STANDPOINT

Prompt Log

Companion to the Product Definition and Market Teardown · Prepared by Katlego Malaka (KayJay) · 10 September 2026

Why This Document Exists

The Build Challenge brief specifically calls out a habit it wants interns to demonstrate: using ChatGPT (or another AI) to help write the prompt before handing it to the builder, rather than typing "make it better" thirty times. This document is the record of that discipline for Standpoint: the exact prompts that produced the schema, the review gate, and the generation flows, in the order they were used, with a short note on why each one is structured the way it is.

Every prompt below is reproduced verbatim from what was actually sent to Claude Code or is currently live in the deployed Edge Functions. Nothing here is reconstructed after the fact.

The Prompting Method

The build brief follows the formula the Build Challenge recommends: ROLE + CONTEXT + OBJECTIVE + USER + FUNCTIONALITY + CONSTRAINTS + OUTPUT. Concretely:

- ROLE / CONTEXT: what Standpoint is and how it differs from every competitor (position, not post, is the unit of work)
- OBJECTIVE / CONSTRAINTS: five non-negotiable rules, stated as architecture, not preference, plus an explicit do-not-build list
- FUNCTIONALITY: the full data model and a phase-by-phase build order, each phase ending at a checkpoint that has to be demonstrated before the next one starts
- OUTPUT: the brief explicitly asks for a build plan and a CLAUDE.md before any code, so the agent's own understanding could be checked and corrected before implementation began

This is also the sprint's own instruction turned back on itself: research and product definition first, prompt second, build third.

Prompt 1 : The Build Brief

SENT TO CLAUDE CODE, IN AN EMPTY REPOSITORY, BEFORE ANY CODE EXISTED

Why structured this way: the rules section comes before the stack section on purpose. An agent that reads the schema first tends to treat the constraints as an afterthought to add later; putting the non-negotiables first, in plain language, before a single table name appears, made them impossible to skip. The phase/checkpoint structure exists so the agent could not silently skip Phase 4 (the review gate, the actual product) in favour of the visually satisfying parts (UI, dashboard).

```
You are building a product called Standpoint. Read this whole brief, then
produce a build plan and a CLAUDE.md before writing any code. Do not start
implementing until I approve the plan.

WHAT IT IS
A LinkedIn positioning system, not a post generator. Every competitor treats
the POST as the unit of work. Standpoint treats the POSITION as the unit of
work and the post as a subordinate artifact. The user first declares what
they stand for, and every draft is scored against that before it can be
published.

NON-NEGOTIABLE PRODUCT RULES
```

```
These are architectural, not preferences. Enforce them in the schema and in
foreign keys, not just in the UI.
1. A draft cannot exist without an idea. An idea cannot exist without a
   pillar. A pillar cannot exist without a stance. Enforce with FK constraints.
2. A user cannot create a pillar until their brand profile has at least one
   contrarian belief.
3. Pillars are capped at three. Enforce server side.
4. The publish action is blocked unless the draft's most recent review
   result has verdict PUBLISH. Enforce server side, not by disabling a button.
5. The product must never store, fetch or display impressions, likes,
   reactions or follower counts. If you find yourself adding such a column,
   stop and ask me.

STACK
- React + Vite + TypeScript + Tailwind
- Supabase: Postgres, Auth, Row Level Security, Edge Functions
- All LLM calls go through a single Supabase Edge Function. The API key
  lives in Supabase secrets and must never appear in client code, in the
  repo, or in any committed file. Add .env to .gitignore in the first commit.
- Model calls: use the Anthropic Messages API.

DATA MODEL
users(id, email)
brand_profiles(id, user_id, role_title, icp_title, icp_industry, icp_pains jsonb)
stances(id, profile_id, statement, type['core'|'contrarian'], evidence)
anti_positions(id, profile_id, topic, reason)
pillars(id, profile_id, name, stance_id, target_pain)
ideas(id, pillar_id, angle, angle_type['contrarian'|'framework'|'evidence'|'story'], target_pain)
drafts(id, idea_id, body, status['writing'|'reviewed'|'published'])
review_results(id, draft_id, scores jsonb, verdict, reasons jsonb, fixes jsonb, created_at)
published_posts(id, draft_id, linkedin_urn, published_at)

RLS on every table: auth.uid() = user_id, traversing the FK chain where the
table has no direct user_id.

BUILD ORDER
Follow this order exactly. Stop at each checkpoint and tell me it passes
before continuing.

Phase 1 - Schema and auth.
Supabase project, all tables, all RLS policies, email auth, plus a seeded
demo account with a password so the app can be demoed without email delivery.
CHECKPOINT: a logged-in user can read only their own rows. Prove it.

Phase 2 - Skeleton flow, no AI.
Every screen wired end to end with manually entered content saved to the
database: brand profile form, pillars, ideas, draft editor. No LLM calls yet.
CHECKPOINT: data moves through the full chain and survives a page refresh.

Phase 3 - The Edge Function.
One function, one secret, all model calls routed through it. Returns JSON.
CHECKPOINT: it returns a valid response to a curl request.

Phase 4 - The Review Gate.
This is the core of the product. I will supply its system prompt separately.
It must be a SEPARATE model call from draft generation, with its own
adversarial prompt, and it must be capable of returning REJECT.
CHECKPOINT: a deliberately generic draft returns REJECT and publishing is
blocked server side.

Phase 5 - Generation.
Profile extraction from the interview, pillar generation, idea generation.
```

```
These may be stubbed with sensible defaults if time runs short. The gate
may not.

Phase 6 - Dashboard.
Pillar coverage balance, substitutability trend over time, audience problem
coverage, stance advancement rate. All computed from our own tables.

DO NOT BUILD
Scheduling, carousels, image generation, a viral post library, engagement
automation, auto-comments, auto-DMs, browser extensions, team seats or
client workspaces. Each was deliberately cut. Do not add them, and do not
add placeholder UI for them.

LINKEDIN API
Publishing uses Sign In with LinkedIn (OIDC) and Share on LinkedIn
(w_member_social scope). Both are self-serve for new developers. Member-level
post analytics requires partner status we do not have, so do not attempt to
read engagement data. If OAuth is not configured, publishing must fail with
an honest message, never a fake success.

UI
Restrained and editorial. Deep navy #0F1E2E, accent #1F6F8B, caution amber
#A45C1E, pass green #2F6B4F, surface #F7F9FA, border #E4EAEE. Inter.
Rejection is amber, never red: a rejected draft is an editorial judgment,
not a system error. Design empty states explicitly for profile, pillars,
ideas and dashboard. The draft editor must keep the pillar, the stance the
draft must advance, and the active anti-positions visible at all times,
including on mobile at 390px.

START BY
Writing the build plan and CLAUDE.md. Ask me anything ambiguous before
you begin.
```

Verbatim, first message of the build session.

Iteration : The Provider Pivot

The brief above specifies the Anthropic Messages API. Partway through the build, a follow-up instruction changed the provider to Groq's OpenAI-compatible API, to avoid a paid API bill during the sprint. Because the build brief mandated a single Edge Function as the only thing allowed to call an LLM provider, this pivot touched exactly one file (llm-gateway/index.ts) and required no change to the review gate, the generation prompts, or any client code. That constraint, written into the very first prompt, is what made a mid-build provider swap a non-event instead of a rewrite. This is the sprint's own ITERATE step, in miniature.

Prompt 2 : The Review Gate System Prompt

SUPABASE/FUNCTIONS/REVIEW-DRAFT/REVIEWPROMPT.TS : SUPPLIED AT PHASE 4, AFTER PHASES 1-3 WERE CHECKPOINTED

Why structured this way: the prompt opens by revoking the one instinct every generation prompt has, to be helpful. "You are not a writing assistant" and "you have no investment in it passing" exist to stop the model from defaulting to encouragement. The scoring rubric is deliberately mechanical (numeric thresholds, verdict rules applied in a fixed order) so the verdict is reproducible and auditable rather than a vibe, which matters because publish_draft() trusts this output completely.

```
You are a reputation critic. You are not a writing assistant and you are not
helpful. Your only job is to judge whether a LinkedIn draft is worth the
author attaching their name to.

You did not write this draft. You have no investment in it passing.
```

```
You will receive:
- The author's belief graph: domains, core stances, contrarian beliefs,
  anti-positions, proprietary frameworks, and their ICP's stated problems.
- Their previously published posts.
- One draft.

Score the draft 0-100 on each of these:

pillar_fit - Does it reinforce one of the declared domains, or drift?
stance_alignment - Does it advance a specific declared stance? Name which.
  If none, score 0.
substitutability - Could 500 other people in this industry have published
  this identical post? HIGHER IS WORSE. A post with no author-specific
  claim, number, case or experience scores above 85.
evidence_specificity - Is there a concrete number, named case, or lived
  detail, or only assertion?
ai_slop_risk - Does it match generic AI shapes: listicles, "In today's
  rapidly evolving landscape", one-line-per-paragraph broetry, or an
  engagement-bait question at the end? HIGHER IS WORSE.

Also determine:
contradicts_post_id - the id of any published post this draft contradicts,
  or null.
anti_position_hit - the anti-position violated, or null.

Verdict rules, applied in order:
- REJECT if anti_position_hit is not null.
- REJECT if stance_alignment < 30 OR substitutability > 75.
- REVISE if evidence_specificity < 40 OR ai_slop_risk > 60 OR
  contradicts_post_id is not null.
- Otherwise PUBLISH.

Return ONLY this JSON. No preamble, no markdown fences.

{
  "scores": {
    "pillar_fit": 0,
    "stance_alignment": 0,
    "substitutability": 0,
    "evidence_specificity": 0,
    "ai_slop_risk": 0
  },
  "stance_advanced": null,
  "contradicts_post_id": null,
  "anti_position_hit": null,
  "verdict": "REJECT",
  "reasons": ["one short sentence per score that drove the verdict"],
  "fixes": ["specific, actionable, referencing the author's own material"]
}

Rules for your output:
- Never praise. Explain.
- Every fix must reference something in the author's belief graph or
  published archive. "Add more detail" is a useless fix. "Replace point 4
  with the nine-week approval cycle you measured across 14 migrations" is
  a useful one.
- A well-written draft that argues nothing is still a REJECT. Fluency is
  not the standard. Being worth reading is.
```

Verbatim, live in the deployed review-draft Edge Function.

The user message this system prompt actually receives is assembled server-side from the author's real data, not typed by hand:

```
{
  "belief_graph": {
    "role_title": "...", "icp_title": "...", "icp_industry": "...",
    "icp_pains": ["..."], "domains": ["..."],
    "core_stances": [{ "statement": "...", "evidence": "..." }],
    "contrarian_beliefs": [{ "statement": "...", "evidence": "..." }],
    "anti_positions": [{ "topic": "...", "reason": "..." }]
  },
  "this_draft_pillar": "...",
  "this_draft_stance": { "statement": "...", "type": "core|contrarian" },
  "published_posts": [{ "id": "...", "body": "..." }],
  "draft": { "body": "..." }
}
```

Shape of the payload built in review-draft/index.ts from brand_profiles, stances, anti_positions and published_posts.

Prompt 3 : Profile Extraction

SUPABASE/FUNCTIONS/GENERATE-CONTENT/INDEX.TS : PHASE 5

Why structured this way: the single most important line is the instruction never to fabricate a contrarian belief the interviewee did not state. Rule 2 of the product (no pillar without a contrarian belief) only holds if this extraction step cannot manufacture one to satisfy the gate. The prompt is written to fail closed, returning an empty array rather than inventing content, whenever the source material is thin.

```
You are a positioning analyst. You extract structured brand-profile data
from a founder's raw interview answers -- you do not invent beliefs they
did not state.

Return ONLY this JSON, no markdown fences, no preamble:
{
  "role_title": "string",
  "icp_title": "string",
  "icp_industry": "string",
  "icp_pains": ["short phrase", "short phrase"],
  "stances": [
    { "statement": "string", "type": "core", "evidence": "string or null" },
    { "statement": "string", "type": "contrarian", "evidence": "string or null" }
  ],
  "anti_positions": [{ "topic": "string", "reason": "string" }]
}

Rules:
- Include at least one stance with type "contrarian", derived specifically
  from the interviewee's stated contrarian belief. Never fabricate a
  contrarian belief they did not express.
- icp_pains are short phrases (2-5 words), not full sentences.
- If the interviewee gave no anti-positions, return an empty array -- do
  not invent one.
```

Verbatim, live in the deployed generate-content Edge Function.

Prompt 4 : Pillar Suggestion

SUPABASE/FUNCTIONS/GENERATE-CONTENT/INDEX.TS : PHASE 5

Why structured this way: stance_index constrains the model to choose from stances that already exist rather than invent new ones, so a generated pillar passes through exactly the same foreign-key and trigger constraints as a hand-typed one. There is no generation-only bypass path.

```
You are a positioning strategist. Given a set of declared stances (each
with an index) and a list of the audience's stated pains, propose pillars
-- domains this person will consistently write about, each one advancing a
specific declared stance.

Return ONLY this JSON, no markdown fences, no preamble:
{ "pillars": [{ "name": "string", "stance_index": 0, "target_pain": "string" }] }

Rules:
- stance_index must be an index into the provided stances array -- never
  invent a stance.
- target_pain should be drawn from (or closely adapted from) the provided
  ICP pains.
- Propose exactly the number of pillars requested, no more.
```

Verbatim, live in the deployed generate-content Edge Function.

Prompt 5 : Idea Suggestion

SUPABASE/FUNCTIONS/GENERATE-CONTENT/INDEX.TS : PHASE 5

Why structured this way: the angle_type enum forces coverage across contrarian, framework, evidence and story angles rather than four variations on the same idea, and the closing rule against generic topic labels ("Tips for better deploys") exists because that failure mode showed up during testing and was prompted against directly rather than patched after the fact in code.

```
You are a content strategist. Given one pillar (its name and the stance it
advances) and the audience's stated pains, propose post ideas -- specific
angles, not generic topics.

Return ONLY this JSON, no markdown fences, no preamble:
{ "ideas": [{ "angle": "string", "angle_type": "contrarian|framework|evidence|story", "target_pain":
"string" }] }

Rules:
- angle_type must be exactly one of: contrarian, framework, evidence, story.
- Propose one idea per angle_type (4 total) unless fewer are requested.
- Each angle must be a specific, arguable claim or story premise -- never a
  generic topic label like "Tips for better deploys".
```

Verbatim, live in the deployed generate-content Edge Function.

What This Demonstrates

- Functionality before UI: every generation prompt above was written and reviewed before any screen that displays its output
- Specific, not random, prompting: no prompt in this log is a variation of "make it better"; each one encodes a rule the product depends on
- Edge-case awareness written into the prompt itself, not bolted on after: refusing to fabricate a contrarian belief, refusing to invent a stance index, refusing to praise a draft that argues nothing
- The review gate and the generation prompts are provably separate system prompts, calling the same gateway but never sharing a call, so a generation model can never grade its own work